

Designed by Remus Teodorescu 2026

LECTURE 11.1

Vision-Based Navigation

Remus Teodorescu

With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

NEED · RULES · NETWORKS



Cars, warehouse robots, underwater vehicles and humanoid robots all steer by what a camera sees, and all answer two questions: where is the path, and how do I steer onto it? The lecture answers both with fixed rules, then with a convolutional network, its training frames and the GPU that makes it fast enough.

Learning Objectives

SECTIONS, IN ORDER

1. **The need** — vehicles and robots that steer by camera
2. **Conventional vision** — fixed rules, and where they fail
3. **Deep learning** — a network that steers, its data, its GPU

Two questions run through the lecture: where is the path, and how do I steer onto it? Each is answered first in words, then in mathematics - by fixed rules, then by a network - on Ex_11's virtual JetRacer, where the true position of the line is known.

READ, AND WHAT EACH ANSWERS

UDL ch. 1	where is deep learning applied?
UDL ch. 10	why do images need convolutions?
UDL ch. 8	what must the training data show?

Recap of L10.2 Electrolysers

1 – 2 · A NETWORK FOR THE FUEL CHANNEL

1 **How can one network learn the steam fraction, temperature and current density along the channel at every operating point?**

The network takes the position along the channel, the cell voltage and the inlet temperature, and returns the three fields. It is trained over 1.15 to 1.35 volts and 750 to 850 degrees until the three laws hold at points over the channel and the operating range; the inlet values are built into the form of the solution, and the current is kept below the limiting current. Any operating point is then read off, not solved again. *Liu ch. 2.2; Ex_10.2 section 5*

2 **Why does the local current make the channel harder to solve, and how is the network's answer checked?**

The current has no equation of its own along the channel: at every point it is the root of the charge transfer at the local steam fraction and temperature, so no field can be solved first. The network takes it as a third residual, in units of 50 millivolts so it weighs like the balances. Scored against a boundary-value solver at twelve operating points, it is within 0.0022 of the steam fraction and 0.19 kelvin, three times closer than finite volumes on 50 cells. *Ex_10.2 sections 3 – 5*

3 – 4 · THE OPTIMAL VOLTAGE

3 **How can the optimal cell voltage be found with a network, and what does the gradient save?**

The cost of an hour, the electricity bought less the hydrogen sold plus the share of the stack aged, is read from the network's fields, with the 10 kelvin limit as a penalty. Autograd through the network gives its derivative in the voltage, and gradient descent finds 1.250 volts in seconds; without the limit, 1.167 volts, where the cell cools 25 kelvin. A sweep of 81 solver runs agrees. The prices and the ageing law are assumptions, so the optimum is conditional on them. *Liu ch. 5; Ex_10.2 section 6*

4 **Why does a network help to optimise the operation of the cell over its lifetime, and what should not be claimed from it?**

Over a lifetime the decision is a trajectory of voltage and temperature: what produces more now ages the cell faster. A solver gives the cost, but its gradient costs one solve per variable per step, or an adjoint built by hand; a trained network gives every gradient by automatic differentiation. The degradation constants here are assumed, and published rates span an order of magnitude, so a predicted lifetime ranks schedules; it is not a number to promise. *Liu ch. 5*

Autonomous Vehicles and Robots

Vision-based navigation — steering a vehicle or a robot from what its camera sees: where the free path is, where it leads, and what is in the way. The camera is cheap and carries the most information of any sensor, but turning its pixels into a decision is the hard part.

FOUR OF THEM, AND THE LOOP THEY SHARE



robotaxi



warehouse AGV



humanoid



underwater vehicle

see one frame → **decide** path, obstacles → **act** steer, speed → **in time** fast enough

Photographs: Wikimedia Commons. This lecture is the path; L11.2 is what blocks it.

WHAT MAKES THE CAMERA'S JOB HARD

	specific challenges	beside the camera
road car	night, rain, low sun; 3.6 m blind per 0.1 s	radar, LiDAR, GNSS
robotaxi (Cybercab)	no wheel or pedals: no driver to take over	none: cameras only
warehouse AGV	shiny floors, narrow aisles, people and forklifts	safety LiDAR, floor markers
underwater vehicle	light gone within metres; murky water	sonar, Doppler log, inertial
humanoid robot	camera sways; stairs need depth	depth camera, joint angles

The JetRacer Pro

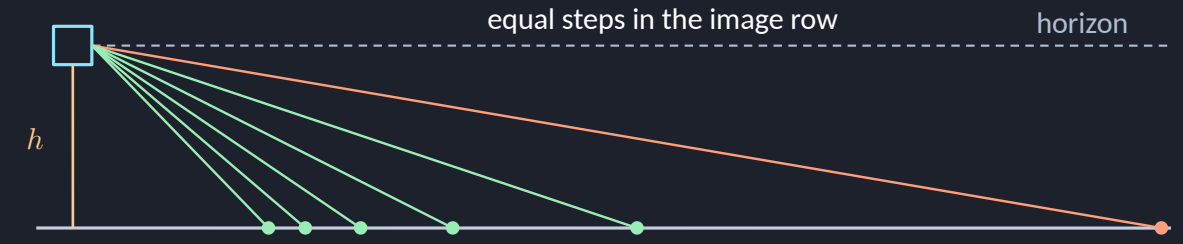
JetRacer Pro — a small car that computes on board: a Jetson Nano runs the network on its GPU, a wide camera is its only sensor, a servo steers and a brushed motor drives. It is the scaled-down case of the machines on the previous slide, with the same loop: see, decide, act.

WHERE A ROW OF PIXELS LANDS ON THE FLOOR

$$v - c_v = \frac{f_v\, h}{Z}, \qquad \Delta Z \approx \frac{Z^2}{f_v\, h}$$

v the image row, c_v the horizon's row, f_v the focal length in pixels
h the camera's height, Z the distance ahead, ΔZ the floor one row covers

A LEVEL CAMERA, SCHEMATIC



THE VERIFIED SPECIFICATION

compute	Jetson Nano 4 GB, 128-core Maxwell GPU
camera	IMX219-160: 8 MP, 160° field of view
steering	Ackermann, 6 kg·cm servo
drive	4WD, RC380 brushed motor
pack	8.4 V, four 18650 cells, 2S2P
encoder	none

docs/hardware/jetracer-pro.md is the course's checked sheet. The camera's height, tilt and field of view are assumed in Ex_11's virtual JetRacer and measured on the car.

The Classical Pipeline

Conventional vision — fixed image-processing steps chosen by an engineer: the frame is reduced to grey levels, the pixels brighter than the floor are kept, the centre of the line is found, and a hand-written rule turns it into a steering command. Nothing is learned from data.

THE DETECTOR, AS EX_11 WRITES IT

$$w_j = \max\left(I_j - \bar{I} - 0.15,\ 0\right), \qquad \hat{x} = \frac{\sum_j w_j x_j}{\sum_j w_j}$$

I_j the brightness of pixel j, $\bar{I}$ the floor's median, x_j its column
w_j its weight, $\hat{x}$ the estimated column of the line

FROM A FRAME TO A COMMAND

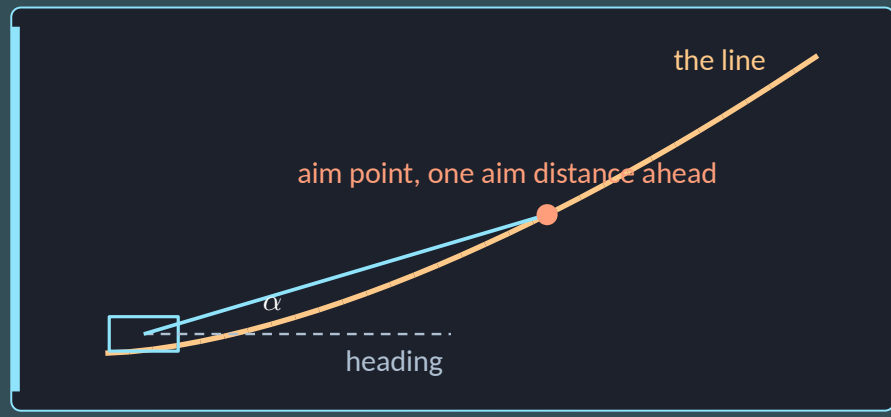
frame grey pixels → **threshold** set by hand → **line** bright centre → **rule** steer to it

every step has a number chosen by an engineer, for one floor and one light

THE RULE THAT STEERS: PURE PURSUIT

$$\delta = \arctan\frac{2L\sin\alpha}{\ell_d}$$

δ the steering angle, α the angle to the aim point
L the wheelbase, 0.20 m, ℓ_d the aim distance, 0.40 m



Wheelbase and aim distance assumed until measured; Ex_11's lap steers this way.

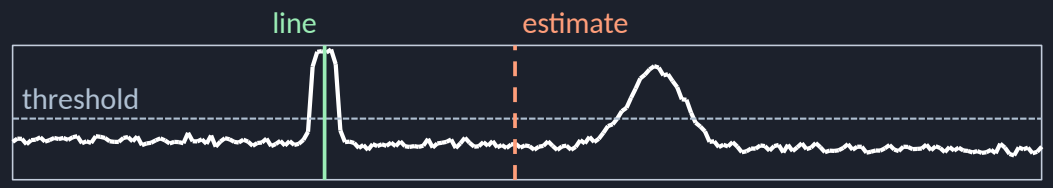
Where Conventional Vision Fails

Robustness — keeping the error within what was agreed when the scene changes: the light, a reflection, a shadow, a new floor, an unseen object. A hand-tuned rule is robust only to the changes its designer thought of.

GLARE PULLS THE ESTIMATE

$$\hat{x} = \frac{W_\ell\, x_\ell + W_g\, x_g}{W_\ell + W_g}$$

W_ℓ, W_g the summed weights of the line and the glare, x_ℓ, x_g their columns



brightness across the frame, column by column

TALK · 2 MIN How would you explain, in your own words, why a rule that works on one floor can fail on another? Then, in the plot above, why does the estimate land between the line and the glare?

THE DETECTOR, 112 × 112, 300 FRAMES

mean error	detector
along the line	0.007
beside the line	0.021
with glare	0.083
the agreed error	0.03

In the frame on the left the glare carries 57 % of the weight, and the estimate moves from column 68 to 109. Bright is all the rule looks for.

Steering From a Frame

Vision policy — a convolutional network that maps one camera frame to one action. It learns its own features from frames whose true answer is known, instead of following rules written by hand. One output between −1 and +1 makes a steerer; two outputs, free or blocked, detect an obstacle.

ONE CONVOLUTION, AS IN L5.1

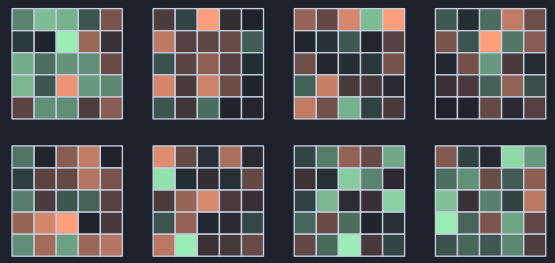
$$y_{ij} = \sigma\Big(\sum_{m,n} w_{mn}\, x_{i+m,\, j+n} + b\Big)$$

x the frame, w the kernel, b its bias, σ the ReLU, y the feature map

THE SAME SUM, TWO WAYS TO CHOOSE THE KERNEL



set by hand
Sobel: an edge



learned from the frames: the eight kernels of the first layer

MEAN ERROR ON 300 HELD-OUT FRAMES

	detector	network
along the line	0.007	0.018
beside the line	0.021	0.035
with glare	0.083	0.055
pixels	112, grey	224, colour
training	none	80 s on a CPU

Four convolutions and two dense layers, trained on the squared error to the true position. Shown glare in training, the network is the better of the two with glare; the price is training, and about twice the time per frame on a CPU.

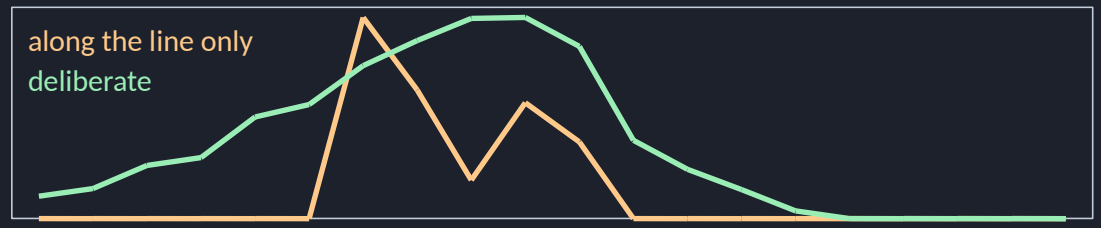
Collect the Data Deliberately

Deliberate collection — choosing the frames the policy will be trained on. The network learns only what the pictures show: the floor, the mount and the light it will drive in, and on purpose the positions the car reaches when it is already going wrong, labelled with the way back.

THE LOSS SEES ONLY THE TRAINING FRAMES

$$\mathcal{L} = \mathbb{E}_{x \sim p_{\text{train}}}[(\hat{y} - y)^2]$$

$\mathcal{L}$ the loss, p_{train} where the line is in the training frames; the car meets p_{drive}



A policy shown only perfect driving has never seen a mistake, so it cannot correct one: small errors add up, lap after lap.

THE SAME NETWORK, TRAINED TWO WAYS

mean error	along only	deliberate
along the line	0.006	0.018
beside the line	0.185	0.035
with glare	0.186	0.055

Trained only along the line the network is 5 times worse exactly where a drifting car needs it. Same network, same number of frames, same training time: only the collection differs.

Why a GPU

GPU — a processor with many small cores that run the same operation on different data at the same time. Every output of a convolution is its own sum of products over a small window, independent of the others, so a GPU computes them all at once.

THE DISTANCE DRIVEN WITHOUT NEW INFORMATION

$$d = \frac{v}{f} = v\,(t_{\text{cap}} + t_{\text{pre}} + t_{\text{net}} + t_{\text{act}})$$

d the blind distance, v the speed, f decisions per second
t the times to capture, preprocess, run the network and act

ONE CONVOLUTION, ALL OUTPUTS TOGETHER

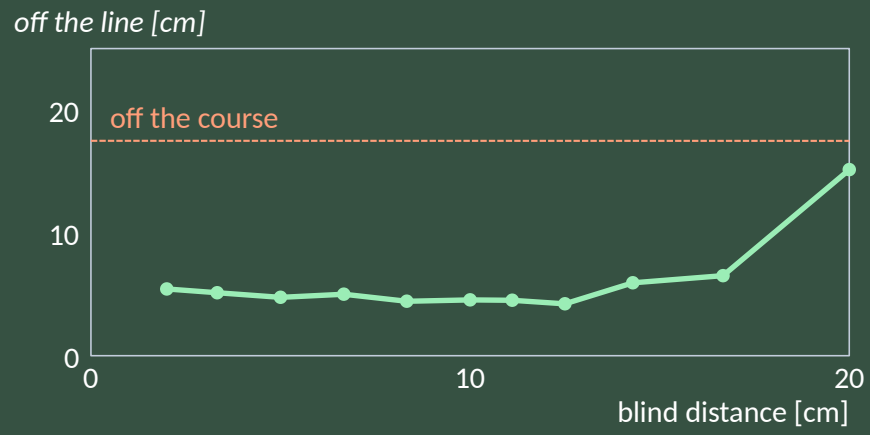


one core per output

the frame, one 3 × 3 window every output at once

TALK · 2 MIN How would you explain, in your own words, why a slightly more accurate but slower network can be a worse driver? Then, in the plot on the right, where does the car start to swing wide?

THE LAP, AND THE WORK PER FRAME



$$\text{MAC} = H_{\text{out}}\, W_{\text{out}}\, C_{\text{out}}\, C_{\text{in}}\, k^2$$

$H_{\text{out}}, W_{\text{out}}$ the output size, $C_{\text{in}}, C_{\text{out}}$ the channels, k the kernel

Ex_11's network	17 million a frame
ResNet-18	1.8 billion, at 224 × 224
Nano GPU	472 GFLOPS at its peak

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
semantic segmentation	which pixels are free to drive on	UDL ch. 10	floors with no line
depth from one camera	distance from a single frame	UDL ch. 10	no range sensor on board
vision transformers	attention over patches of the frame	UDL ch. 12	large, varied data sets
imitation learning	a policy from human driving	UDL ch. 19	road vehicles
reinforcement learning	a policy from reward, by trial	UDL ch. 19	simulators, legged robots

WHY THEY ARE LEFT OUT

Ex_11 follows one taped line with one camera, scored on rendered frames where the true position is known, so none of them is needed to do it. They matter for the full-size machines of the first section.

Exercise 11

ONE STUDY NOTEBOOK, FROM A TO B

- **1 – 3** — the virtual JetRacer, its frames, the detector
- **4** — a network that steers, trained deliberately
- **5 – 6** — obstacles, and the strategies to avoid them
- **7** — the loop rate, the GPU and fewer bits
- **8 – 10** — the report, and the mini projects



the same task on the real car: a demonstration in the lab

THE QUESTION EACH PART ANSWERS

1 – 3	when is a fixed rule enough?
4	what does deliberate data buy?
5 – 6	which strategy reaches B first?
7	how fast must the loop be?

REPORT QUESTION, TO CONCLUDE

Which avoidance strategy reaches B fastest without a collision, and what limits its speed: the errors of the networks, the loop rate, or the distance needed to react?

SUMMARY

Key Takeaways

VISION POLICY

what it is	a CNN from a frame to an action
function	where the line is, a little ahead
train, tune	frames that show the mistakes
vs a fixed rule	no hand tuning, holds under glare
used for	steering cars, robots, vehicles

THE LOOP ON A GPU

what it is	many cores, one operation each
function	all outputs of a convolution at once
train, tune	the whole loop, measured on the car
vs accuracy	judged together, rate first
used for	a short blind distance $d = v/f$

Next: what blocks the path - obstacles, and what to do about them.

Questions

1 – 2 · A NETWORK THAT STEERS

1 **Why can a convolutional network steer from a camera frame where a hand-tuned line detector fails?**

The detector follows fixed steps chosen by an engineer: it keeps the pixels brighter than the floor and steers to their centre, so anything bright is taken for the line. The network learns its own features from frames whose true position of the line is known. On clean frames the detector meets the agreed error; with a patch of glare its error rises to 0.08, while the network, shown glare in training, keeps it at 0.055. *UDL ch. 10; Ex_11 sections 3 – 4*

2 **What must the training frames show for the network to steer a car that is drifting off the line?**

The network learns only what the frames show: the floor, the camera mount and the light it will drive in, the disturbances such as glare, and on purpose the positions the car reaches when it is already going wrong, labelled with the way back. Trained only along the line, the same network is five times worse beside it, which is exactly where a drifting car needs it. Same network, same number of frames: only the collection differs. *UDL ch. 8; Ex_11 section 4*

3 – 4 · THE LOOP ON A GPU

3 **Why is a steering network judged by the rate of the control loop as well as by its error?**

Between two decisions the car drives on its last command with no new information, over a blind distance of its speed divided by the decisions per second. With the network steering the virtual car round the lap it stays within 6 cm up to 17 cm blind and swings 15 cm at 20 cm, so a slightly more accurate network that halves the loop rate is usually a worse car. The whole loop is timed, from capture to command, because the network is only one of its stages. *Ex_11 section 7*

4 **Why does a convolutional network need a GPU to steer in time, and what must be measured on the car?**

Every output of a convolution is its own sum of products over a small window of the frame, independent of the others, so the many cores of a GPU compute them at the same time. A ResNet-18 needs about 1.8 billion multiply-adds for one 224 by 224 frame. The peak rate of the GPU is only a ceiling: the loop rate that sets the blind distance is measured on the car itself, from the frame being captured to the steering command. *UDL ch. 10; Ex_11 section 7*

NEXT

L11.2 — Obstacle Avoidance

What blocks the path: a network that tells free from blocked, the strategies that steer around an obstacle, and the distance the car needs to react.

BEFORE THE NEXT SESSION

read	UDL ch. 10
run	Ex_11's notebook, sections 1 to 4
bring	your detector and network errors
think	what in your frames is an obstacle?

The real JetRacer drives the same task in a lab demonstration, with the throttle limited in software and one hand on the stop.